NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

MARCOS VINICIUS BONFIM ROMÃO

Licenciado em Engenharia Eletrotécnica e de Computadores

PRE-DISSERTATION DATA

5G RADIO ACCESS NETWORK AND CORE

Dissertation Plan

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING

NOVA University Lisbon

Draft: *December 31, 2025*

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

PRE-DISSERTATION DATA

5G RADIO ACCESS NETWORK AND CORE

MARCOS VINICIUS BONFIM ROMÃO

Licenciado em Engenharia Eletrotécnica e de Computadores

Adviser: Rodolfo Oliveira
Full Professor, NOVA University Lisbon

Dissertation Plan
MASTER IN ELECTRICAL AND COMPUTER ENGINEERING

NOVA University Lisbon
Draft: *December 31, 2025*

Contents

List of Figures iii

List of Tables iv

Glossary v

Acronyms vi

Symbols vii

1 Chapter Title 2
- 1.1 Motivation . . . 2
- 1.2 Goals . . . 2
- 1.3 Report Structure . . . 2

2 Related Work 3
- 2.1 Introduction . . . 3
  - 2.1.1 Sensing Technologies . . . 3
- 2.2 Fundamentals of FMCW Operation . . . 5
  - 2.2.1 History of Radar . . . 5
  - 2.2.2 FMCW Radar operation overview . . . 5
  - 2.2.3 Measurements . . . 7
- 2.3 Feature Engineering . . . 11
  - 2.3.1 Time-Domain Features . . . 11
  - 2.3.2 Frequency-Domain Features . . . 13
  - 2.3.3 Feature Engineering . . . 15
- 2.4 Machine Learning . . . 18
  - 2.4.1 Introduction to Machine Learning Techniques . . . 18
  - 2.4.2 Deep Learning . . . 20
- 2.5 State of the Art / Related Works . . . 30

3 Work Plan 34
- 3.1 Task Description . . . 34
- 3.2 Schedule . . . 35

4 Conclusions 37

Bibliography 38

List of Figures

2.1 FMCW Radar Block Diagram Representation. 6
2.2 Linear Sawtooth modulation. 6
2.3 a) Linear sawtooth FMCW transmitted waveform in black. Received signals with delay τ_1 and τ_2 in colour, each corresponding to different targets represented in b). b) Two static targets at different ranges from the radar. 7
2.4 Complex-baseband mixer implementation. 8
2.5 a) Received signal shifted upwards due to Doppler effect of a target moving closer to the radar b) Received signal shifted downwards due to Doppler effect of a target moving away from the radar. 10
2.6 a) Connected Neural Network b) Analogy between biological and artificial neurons. 20
2.7 Sigmoid function plot. 22
2.8 Hyperbolic Tangent function plot. 22
2.9 Rectified Linear Unit function plot. 23
2.10 CNN architecture inspired by [38]. 25
2.11 a) Unfolded RNN architecture b) Folded RNN inspired by [41]. 26
2.12 LSTM architecture (adapted from [41]). 27
2.13 GRU architecture (adapted from [41]). 28
2.14 GAN architecture (adapted from inspired by [46]). 29
2.15 Autoencoder architecture. 30

3.1 Gantt Chart for the timeframe of this thesis work plan. 36

List of Tables

Acronyms

2D-FFT Two-Dimensional Fast Fourier Transform *14)*

ADC analog-to-digital converter *7*, *9*, *14)*
AE Autoencoder *20)*
AI Artificial Intelligence *18)*
ANN Artificial Neural Network *20*, *21*, *31)*
ANOVA Analysis of Variance *33)*

Bi-LSTM Bidirectional Long Short-Term Memory *32)*

CNN Convolutional Neural Network *17*, *20*, *24*, *25*, *32)*
COV Covariance *12)*
CW Continuous wave *5*, *6)*

DL Deep Learning *20)*
DRDT Dynamic Range-Doppler Trajectory *31)*
DSP Digital Signal Processing *7*, *10)*

FFT Fast Fourier Transform *7*, *9*–*14*, *33)*
FMCW frequency-modulated continuous-wave *3*, *5*, *31*, *33)*
FoV Field of View *4)*

GAN Generative Adversarial Network *20*, *29)*
GRU Gated Recurrent Unit *26*, *28)*

HAR Human Activity Recognition *32)*

I in-phase component *8*, *13)*
I/Q in-phase and quadrature components *13)*
IF Intermediate Frequency *7*–*11)*
ISOMAP Isometric Mapping *16*, *17)*

KNN K-Nearest Neighbour *19, 31, 32)*

LDA Linear Discriminant Analysis *16, 17)*
LFM Linear Frequency Modulation *5, 7)*
LPF Low Pass Filter *7, 9)*
LReLU Leaky Rectified Linear Unit *23)*
LSI Latent Semantic Indexing *16, 17)*
LSTM Long Short-Term Memory *26)*

MI Mutual Information *33)*
ML Machine Learning *3, 11, 18–20)*
mRMR minimum Redundancy Maximum Relevance criterion *16, 31)*

PCA Principal Component Analysis *16, 17)*
PP Peak to Peak *12)*
PReLU Parametric Rectified Linear Unit *23)*
PRI Pulse Repetition Interval *6)*
PSE Power Spectral Efficiency *14)*

Q quadrature component *8, 9, 13)*
QEA Quantum-Inspired Evolutionary Algorithm *31)*
QEA-IF Quantum-Inspired Evolutionary Algorithm - Information Factor *31)*

R Range *9)*
RADAR Radio Detection and Ranging *5)*
RDI Range Doppler Image *14)*
RDM Range-Doppler Map *31)*
ReLU Rectified Linear Unit *21–23)*
RF Radio Frequency *5, 30, 33)*
RFE Recursive Feature Selection *33)*
RNN Recurrent Neural Network *20, 25, 26)*
RReLU Randomized Rectified Linear Unit *23)*
RTD Round Trip Delay *9)*

SD Standard Deviation *12, 15)*
SVM Support Vector Machine *19, 32)*

ToF Time of Flight *4, 6–8)*

Var Variance *12)*

VCO Voltage Controlled Oscillator *6*, *8)*

ypeoutNT FILE chapter1.tex

1 Chapter Title

1.1 Motivation

1.2 Goals

1.3 Report Structure

2 Related Work

2.1 Introduction

In order to design a sensing system, it is important to determine the type of sensor to be used. This choice has to address the challenges presented by the environment in which the experience is conducted, as well as the goals of the experiment itself. The evaluation of several types of sensors in Subsection 2.1.1 provides context for the decision to use the frequency-modulated continuous-wave (FMCW) radar for human motion detection in this thesis. Its operation is briefly explained in Subsection 2.2.2.

Machine Learning (ML) is the process of extracting patterns from data to make decisions, and it is a very appealing solution to complex problems that entices researchers from a variety of communities to use it in their projects [1]. However, as technology has advanced, datasets have grown larger and more complex, requiring faster analysis. Raw data must be transformed into meaningful features that represent specific properties of the observations before ML algorithms can be applied. This process, known as *feature engineering*, is crucial to the success of machine learning [2].
Since ML has a wide range of applications, each with different characteristics, there is no standard approach to feature engineering, thus, it is important to experiment with different techniques and observe their effects on model performance. Sections 2.3 and 2.4.1 summarize features engineering and ML topics, respectively.

2.1.1 Sensing Technologies

When it comes to motion and gesture sensing, there are several types of sensors to be considered. They can be divided into wearable and non-wearable sensors. The first can gather data with high resolution from body movements, namely their acceleration, angular velocity, and displacement [3], but they have one critical limitation: their practicality. This is due to the fact that to recognize a gesture, the participant needs to be wearing the said device, [4]. Accelerometers and gyroscopes are examples of wearable sensors. On the

other hand, non-wearable sensors do not have this limitation and can have the upper hand regarding battery life duration [3]. Furthermore, the technology needs to be user-friendly and functional when the user is responsible for operating the equipment. Non-wearable sensors combine those characteristics and can include cameras, radars, infrared, ultrasonic, and laser sensors. A brief overview of these sensing alternatives is discussed in the following paragraphs. **Cameras** have advanced over time in terms of resolution and this qualifies them as an excellent option for sensing systems, since pictures or frames with better quality, almost directly imply a better performance in gesture recognition solutions [5]. However, there is a tradeoff between this and their sensitivity to light conditions and inaptitude to capture images in cases of occlusion. Therefore, most studies using them take place indoors. Not only that but in cases where privacy is valued, cameras might not be the best option since they compromise the identity of the subject.

In the case of **Infrared** sensors, the broad Field of View (FoV) is one of their characteristics. Their drawbacks include that they operate only at small distances and that meteorological conditions can negatively affect their performance.

Ultrasonic sensors can be used to measure an object's distance based on the time interval it takes for an emitted wave to be sent and reflected. They operate on a higher range of frequencies, higher than the range of human hearing, and the waves emitted are faster than the speed of light. Using these types of sensors rules out limitations like colour and transparency of objects, and light and environment conditions such as dust, dirt, or high-moisture [6]. However, the temperature can negatively affect their performance [7], as well as certain softer, smaller or curvier objects.

Using **LiDAR** (Light Detection and Ranging) technology has many advantages, including the precision and accuracy obtained in the distance measurements acquired using a pulsed laser. Using this form of range estimation gives excellent results at larger distances, the only setback is the computational power required to make this solution feasible.

In addition, **Radar** can also be used for sensing, and it is the technology employed for the purpose of this thesis. Distances to objects nearby can be obtained given the Time of Flight (ToF), which estimates the time it takes for a radar wave to be sent and reflected back to the radar. Unlike the previously mentioned, the radar performs great under challenging meteorological and lighting conditions, and it can even work in cases of occlusion, penetrating most materials. It respects the users' privacy and can easily be integrated into objects while remaining discrete and not taking away from their aesthetic appeal. However, object classification in radar-based systems still faces many challenges despite its great potential.

2.2 Fundamentals of FMCW Operation

2.2.1 History of Radar

A radar device can detect obstacles by emitting a Radio Frequency (RF) signal and estimating the time it takes for it to be reflected by an object's surface. By studying the characteristics of the received signal, it's possible to gather information about these objects, such as their speed and the distance at which they stand, hence the name Radio Detection and Ranging (RADAR).
Radar's development path dates back to the 19th century, when Heinrich Hertz realized radio waves suffered some kind of interference due to metallic obstacles in their way. Outing that information amongst the scientific community led to Nikola Tesla stating his theory for that matter. In his words, "When we raise the voice and hear an echo in reply, we know that the sound of the voice must have reached a distant wall or boundary, and must have been reflected from the same. Exactly as the sound, so an electrical wave is reflected, and the same evidence can be used to determine the relative position or course of a moving object such as a vessel at sea" [8].
But it was not until 1904 that a prototype called Telemobiloscope, developed by Christian Hülsmeye was first demonstrated. It was a simple chip capable of detecting objects even in the presence of fog, although it couldn't determine their range. During WWII, radar technology was crucial to the Allies' victory and has since undergone many improvements. The credit goes to the british physicist Robert Watson-Watt and his radio research team, who, in the midst of a project for detecting enemy aircraft, came up with the idea for the first pulsed radar to exist globally [9].
The term RADAR ended up being coined in 1939, and later studies proved that the use of higher frequencies made the radar systems more robust towards weather conditions and that short pulses of radio waves would revolutionize future radar solutions [8].
Ever since, further study of radar as well as several breakthroughs in areas such as electronics and computing made it possible for physical devices to be smaller, portable, more efficient and sophisticated. Today, this technology is embedded in a vast range of use cases, from health care [10, 11] to building, security and surveillance [11], environmental applications [12], aviation, shipping, mapping, and vehicle safety [13].

2.2.2 FMCW Radar operation overview

The FMCW radar operates with Continuous wave (CW), which means it produces a continuous signal, contrary to pulsed radars. Frequency-modulated signals vary their frequency over time, sweeping across a specific bandwidth. Frequency changes are based on the Linear Frequency Modulation (LFM) waveform, which may be linear sawtooth FM, linear triangular FM or segmented FM, although the first one is the most common, and is represented in Figure 2.2. This signal is produced by the waveform generator, as shown

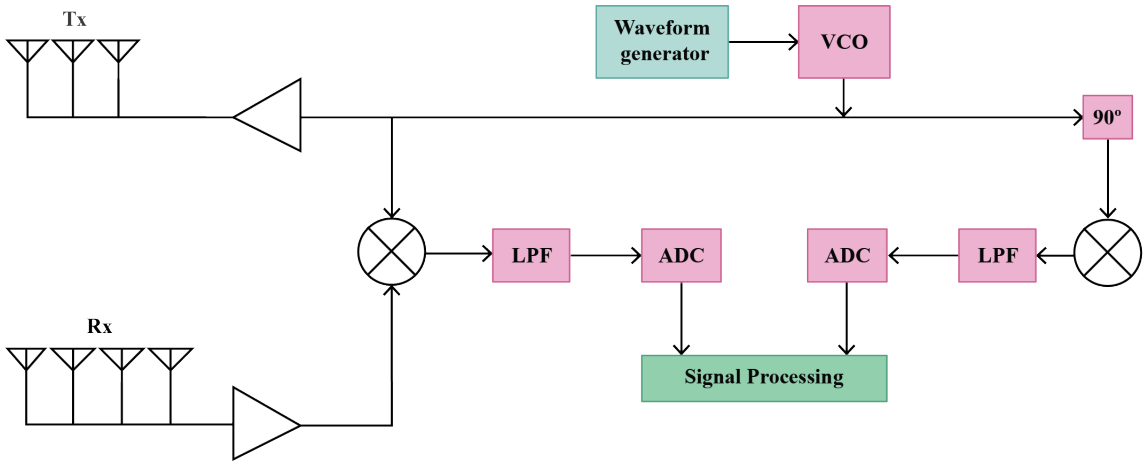

Figure 2.1: FMCW Radar Block Diagram Representation.

in Figure 2.1 where a FMCW radar scheme is presented, and it shapes the transmitted signal's frequency in the Voltage Controlled Oscillator (VCO).

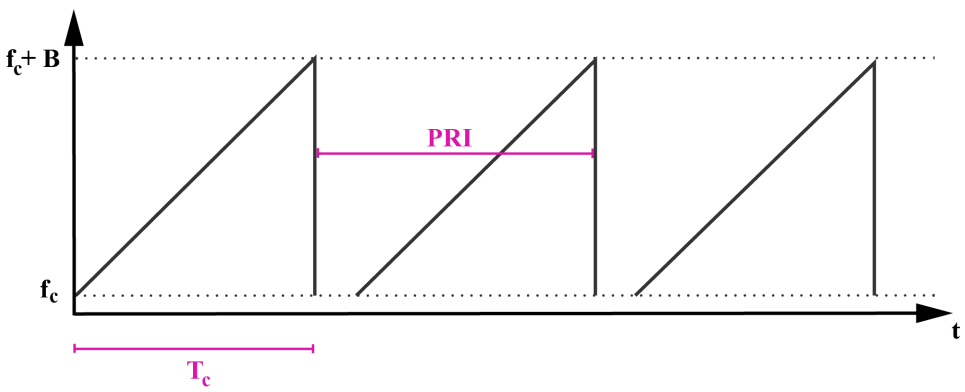

Figure 2.2: Linear Sawtooth modulation.

The bandwidth B is swept periodically during a time period T_c, known as *sweep time*, resulting in multiple transmitted *chirps* over time. It is necessary to assure a delay between chirps, in order to eliminate any potential ambiguities when estimating range and velocity later on. In Figure 2.2, the Pulse Repetition Interval (PRI) encapsulates the period T_c and that delay. Without modulation, it would not be possible to measure ToF in CW radars because there would be no way to distinguish between multiple continuous echoes received as a continuous signal is being transmitted. With modulation, the variation in frequency allows the detection of a delay in the reception of these echoes, since the frequency transmitted is different and known at each instant.

In order to compensate for possible losses along the way, the VCO's signal is amplified before being transmitted. Doing so allows it to reach the desired targets, thus affecting range, and to be reflected with enough power to be detected. Furthermore, the received signal is amplified to make it strong enough for processing, improving the ability to

pinpoint subtle variations in frequency, and allowing distance and velocity to be calculated more accurately later on. The linear sawtooth FMCW waveform transmitted by the radar and an example of two echos received as a result of two different targets' reflections can be observed in Figure 2.3.

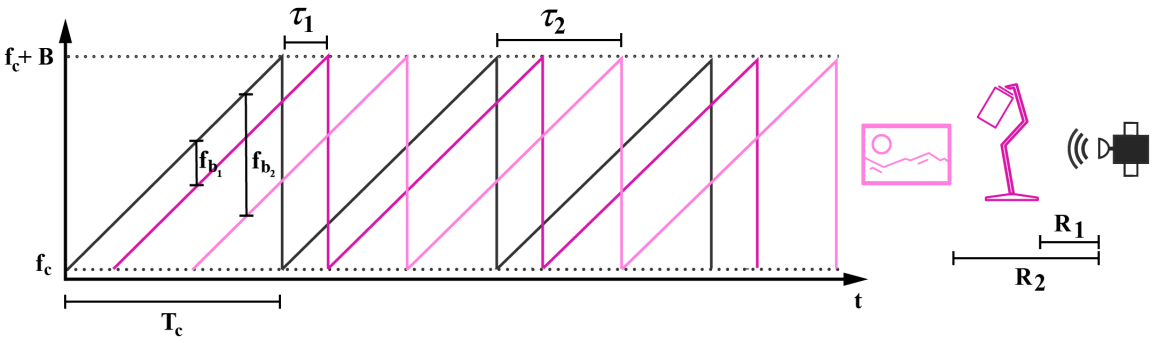

Figure 2.3: a) Linear sawtooth FMCW transmitted waveform in black. Received signals with delay τ_1 and τ_2 in colour, each corresponding to different targets represented in b). b) Two static targets at different ranges from the radar.

By mixing the transmitted and received signals it is possible to obtain the Intermediate Frequency (IF) signal or beat signal, which reflects the difference in frequency of the two caused by ToF. This operation is performed recurring to a mixer. As we can see from the diagram in Figure 2.1, the IF signal then passes through a Low Pass Filter (LPF), followed by an analog-to-digital converter (ADC), which will transform the analog signal to a discrete digital signal, at a fixed rate of samples. The digital signal is passed to the Digital Signal Processing (DSP) block, which will then apply a Fast Fourier Transform (FFT). The output of this operation results in a beat frequency spectrum, that can then be used by the radar system to detect and track targets, and measure range and velocity.

2.2.3 Measurements

2.2.3.1 Distance measurements

For the purpose of this thesis, a sawtooth LFM will be considered. It is possible to define the frequency of each chirp as

$$f(t) = f_c + \frac{B}{T_c}t, \tag{2.1}$$

with f_c corresponding to the carrier frequency, B the swept bandwidth of each chirp and T_c the period of each chirp. Note that the B/T_c quotient is the frequency sweep rate or chirp rate, represented by the slope of the sawtooth signal.

In telecommunications, the standard way to represent a signal is with a cosine function, and a standard signal with an amplitude value of 1 and a null phase can be described as

$$A(t) = Acos(2\pi ft). \tag{2.2}$$

Given the block diagram in Figure 2.1, the transmitted signal can be written as

$$S_{Tx}(t) = Acos(2\pi(f_c t + \frac{B}{2T_c}t^2)). \tag{2.3}$$

Accounting for the delay caused by ToF, represented by τ, it is possible to write the echo of the transmitted signal, the received signal, as

$$S_{Rx}(t) = Acos(2\pi(f_c(t-\tau) + \frac{B}{2T_c}(t-\tau)^2). \tag{2.4}$$

The IF signal is a sinusoidal wave of frequency f_b, beat signal frequency, a result of the subtraction of frequency f_{Tx} and f_{Rx}. Therefore, the beat signal frequency is given by

$$f_b(t) = f_{Tx}(t) - f_{Rx}(t). \tag{2.5}$$

To obtain the mentioned IF signal, it is necessary to submit the transmitted and received signal to a mixer, which can operate based on a real baseband or a complex-baseband implementation. The first one uses a real mixer and a real baseband architecture, using only the in-phase component (I) component of the transmitted signal. Therefore, it only uses one ADC chain, and the final output it produces has a more significant performance loss than complex-baseband implementation.
The complex-baseband implementation uses a quadrature mixer, using both I component and quadrature component (Q) of the signal produced by the VCO, consequently needing two ADC chains. Although it is more expensive than the first solution, the output signal has a better noise-figure ratio [14]. In Figure 2.4 we can see the representation of complex-baseband implementation, which will be used in the scope of this thesis.

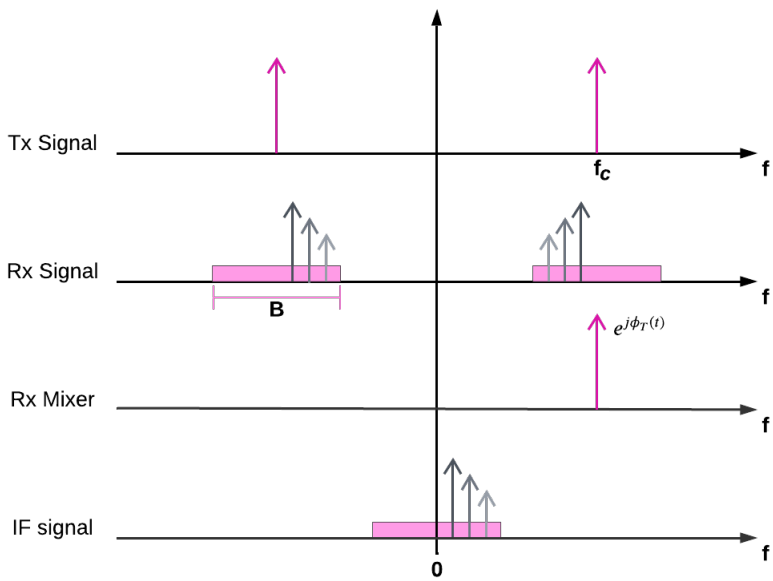

Figure 2.4: Complex-baseband mixer implementation.

The I component, S_{Tx}, is mixed with the received signal, S_{Rx}, as can be described by the following equation, based on [15]

$$S_{Tx} \cdot S_{Rx} = \frac{A}{2} cos\left(2\pi\left(f_c(2t-\tau) + \frac{B}{T_c}\left(t^2 - t\tau + \frac{\tau^2}{2}\right)\right)\right) + \frac{A}{2} cos\left(2\pi\left(f_c\tau + \frac{B}{T_c}\left(t\tau - \frac{\tau^2}{2}\right)\right)\right). \tag{2.6}$$

For the second chain of LPF and ADC, the Q component is mixed with the received signal, originating the following expression

$$S_{Tx}^{90} \cdot S_{Rx} = \frac{A}{2} sin\left(2\pi\left(f_c(2t-\tau) + \frac{B}{T_c}\left(t^2 - t\tau + \frac{\tau^2}{2}\right)\right)\right) + \frac{A}{2} sin\left(2\pi\left(f_c\tau + \frac{B}{T_c}\left(t\tau - \frac{\tau^2}{2}\right)\right)\right). \tag{2.7}$$

Based on [15], the first term of both signals will be cancelled either because it is beyond the mixer's cutoff frequency or because LPF is applied. So, it is possible to write the IF signal as

$$S_{IF}(t) = \exp\left(j2\pi\left(\frac{B\tau t}{T_c} + f_c\tau - \frac{B\tau^2}{2T_c}\right)\right). \tag{2.8}$$

Finally, it is possible to obtain the beat signal by differentiating the instantaneous phase term of the IF signal, obtaining

$$f_b = \frac{d}{dt}\left[\left(\frac{B\tau t}{T_c} + f_c\tau - \frac{B\tau^2}{2T_c}\right)\right] = \frac{B\tau}{T_c}. \tag{2.9}$$

The Range (R) of the target when τ is known represents the Round Trip Delay (RTD). Given the speed of light $c = 299792458\ m/s$ at which the transmitted wave travels, R is defined as

$$R = \frac{c\tau}{2}. \tag{2.10}$$

Given the relation established in (2.9), we can also rewrite (2.10) to obtain the range based on the beat frequency of the IF signal, given by

$$R = \frac{c f_b T_c}{2B}. \tag{2.11}$$

If several targets are considered, a Range FFT can be applied for each one to obtain the beat signal frequency. It is also important to note that with multiple targets, range resolution is an important performance parameter that measures the smallest range difference that the radar system can detect, and can be described as

$$\Delta R = \frac{c\Delta f_b T_c}{2B} = \frac{c}{2B}. \tag{2.12}$$

2.2.3.2 Velocity measurements

In addition to range, it is also possible to obtain a moving target's velocity using the FMCW radar. In this case, there will be a frequency shift called the Doppler shift, f_d,

which according to [16] can be determined by

$$f_d = \frac{2v f_c}{c}. \tag{2.13}$$

The received wave will be shifted upwards if the target is moving towards the radar, or downwards in the opposite case, as we can see in Figure 2.5.

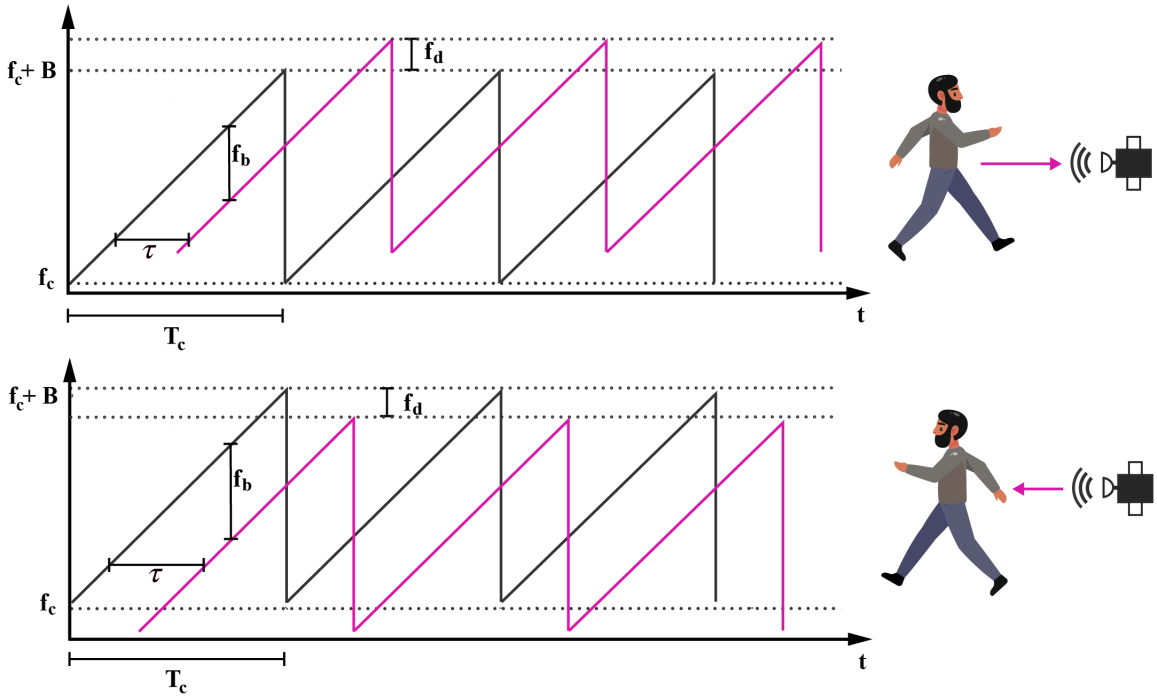

Figure 2.5: a) Received signal shifted upwards due to Doppler effect of a target moving closer to the radar b) Received signal shifted downwards due to Doppler effect of a target moving away from the radar.

In the previous section, it was established that the beat frequency was dependent on the received signal's delay, and in this section it is shown how the Doppler effect also has an impact on that frequency. It is possible to write the received signal's frequency as

$$f_{Rx} = f_c + \frac{B}{T_c}(t - \tau) + f_d. \tag{2.14}$$

The received signal resultant of the reflection on a moving target can be written as

$$S_{Rx} = \cos\left(2\pi\left((f_c + f_d)(t - \tau) + \frac{B}{Tc}\left(\frac{t^2}{2} - \tau t\right) + f_d(t - \tau)\right)\right). \tag{2.15}$$

Recalling (2.9) and (2.13), it is possible to obtain the IF signal similarly as before, resulting in the following expression

$$S_{IF}(t) = \exp\left(j2\pi\left(\left(\frac{B\tau}{T_c} + \frac{2v}{c}f_c\right)t + f_c\tau - \frac{B\tau^2}{2T_c}\right)\right). \tag{2.16}$$

As depicted in Subsection 2.2.2, the DSP block will perform FFT operations to the collected data samples in order for it to be possible to obtain range and velocity information.

The first FFT is performed column-wise, making it possible to obtain range information from what it is called a Range Map or Range Profile, a matrix with dimension N x M, where M is the number of chirps transmitted and N is the number of samples. This first transformation can therefore also be denoted as Range FFT.
However, according to [16], in the case of moving targets, the amplitude and frequency of each of the IF signals will be similar, and therefore not relevant in determining the targets' velocity. On the other hand, the phase of IF signals will suffer slight changes during time, and by applying a row-wise FFT to the Range Map previously mentioned, it is possible to obtain a Doppler Spectrum or Range-Doppler profile that contains relevant information for velocity measurement. This second transformation can also be denoted as Doppler FFT.
As stated in the previous paragraph, a phase difference can be observed between IF signals, and it is described as in (2.17). Based on that expression, it is also possible to obtain radial velocity as it is defined in (2.18).

$$\Delta\omega_v = 2\left(\frac{2\pi v T_c}{\lambda}\right), \tag{2.17}$$

$$v = \frac{\lambda \Delta\omega_v}{4\pi T_c}. \tag{2.18}$$

Given (2.18) and accounting that phase resolution is limited to $\Delta\omega_v = 2\pi/N_p$ [16], and the velocity resolution can be described as

$$\Delta V = \frac{\lambda}{2NT_c}. \tag{2.19}$$

2.3 Feature Engineering

In general, a feature is a distinctive aspect of an observed phenomenon that sets it apart in some way. In the scope of ML, a feature is a measurable attribute of a phenomenon under study. The selection of appropriate features for use in a machine learning model is crucial for achieving good performance, as the model's ability to learn and make accurate predictions is reliant on the features provided to it.

Features used in ML can be classified into various categories based on their characteristics, such as whether they are numeric or categorical, and how they are obtained, such as being directly extracted from a data series or derived from other features. Features can also be divided based on their domain, and the subsequent paragraphs will discuss both time-domain and frequency-domain features, which are both numerical.

2.3.1 Time-Domain Features

Time domain features are derived from time series data, which consist of a series of data points collected over a period of time. The signals received by the radar are in the time

domain before being submitted to a FFT, and therefore it is possible to extract several features such as:

Peak to Peak value (PP) The Peak to Peak (PP) value of a signal is the difference between the highest and lowest points of a signal over a period of time. It is often used to characterize the overall size or amplitude of the signal $x(t)$, and its expression is given by:

$$PP = |max(x(t)) - min(x(t))|. \tag{2.20}$$

Standard Deviation (SD) Standard Deviation (SD) is a statistical measure used to quantify the amount of variation or dispersion in a dataset, and it is the square root value of Variance (Var). It is useful in determining the extent to which the data values in a dataset deviate from the average value of the dataset, meaning the smaller the SD value the closer the data is to the mean of the dataset. It can also be used to compare the dispersion of different datasets, and it is a more intuitive measure than Var. SD is described as

$$SD = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \overline{x}^2)}. \tag{2.21}$$

Covariance (COV) can be useful to assess the relationship between two variables, measuring how they change in respect to one another. A negative covariance means the growth of one variable translates into the decrease of the other. Conversely, a positive covariance reflects two variables that grow alongside one another. When the covariance is zero, there is no established relation between the two. Covariance (COV) is determined by

$$COV(X,Y) = \frac{1}{N}\sum_{i=1}^{N}(x_i - \overline{x})(y_i - \overline{y}). \tag{2.22}$$

Root mean square (RMS) is often used to describe the effective or average power of a time-varying signal and is also denoted as the quadratic mean, as it's defined by the square root of the mean of the squares of the dataset's values, as follows

$$RMS = \sqrt{\frac{1}{N}\sum_{i=1}^{N}x_i^2}. \tag{2.23}$$

N-th Raw Moment (μ'_n) Raw moments are meausures related to the shape and tendency of a distribution. The first four raw moments are mean, variance or standard deviation, skewness and Kurtosis value, respectively. The expression for the n-th raw moment is given by

$$\mu'_n = \frac{1}{N}\sum_{i=1}^{N}x(i)^n, \tag{2.24}$$

where x_i represents a sample of the dataset, and N represents the size of the fixed window of samples.

N-th Central Moment (μ_n) Central moments are normalized versions of raw moments and are generally preferred because they are more robust to changes in scale compared to raw moments. They are defined as

$$\mu'_n = \frac{1}{N}\sum_{i=1}^{N}(x(i) - \mu'_1)^n, \tag{2.25}$$

where x_i represents a sample of the dataset, N represents the size of the fixed window of samples, and μ'_1 represents the first raw moment, or in other terms, the mean value.

Magnitude (M) and Phase (Φ) of the I and Q signals are defined as

$$M = \sqrt{I^2 + Q^2}, \tag{2.26}$$

$$M = \arctan\left(\frac{Q}{I}\right). \tag{2.27}$$

Discrete Cross-Correlation In signal processing, it is a common measure of the similarity between two signals. It can be useful to identify patterns in data, and understand the relationship between different signals. It can be described as

$$\hat{r}_{xy}(k) = \frac{1}{M}\sum_{m=0}^{M-1} x(m)y^*(m+k), \tag{2.28}$$

where M is the length of the cross-correlation and $x(m)$ and $y^*(m)$ are the amplitude and conjugate amplitude of the sampled signals at time m.

Crest Factor is a measure of the peak-to-average power ratio of a signal. It is often used to evaluate the amount of distortion that a system or process introduces when processing a signal, it is defined as

$$CRF = \frac{PP}{RMS}. \tag{2.29}$$

2.3.2 Frequency-Domain Features

As previously stated, FFT will be applied to the in-phase and quadrature components (I/Q) signals, enabling the extraction of frequency-domain features such as the ones we will discuss in this section. It is worth noting that spectral leakage may introduce noise into the spectrum and decrease frequency resolution for the following measurements of range and velocity, but this can be mitigated by selecting an appropriate window size prior to performing the FFT [17]. An alternative method is to use tailored window functions on the signal, such as the Hanning and Hamming windows, the first of which is successful in 95% of cases, according to [18].

Power Spectral Entropy (PSE) This feature plays an important role in designing and optimising wireless communication systems because it relates to the amount of data transmitted over a given bandwidth, describing how the power of a signal is distributed over frequency. The Power Spectral Density function is denoted by $\hat{P}(w_i)$ and is defined in (2.30). Its distribution function is given by 2.31. With those two functions, it is possible to define Power Spectral Efficiency (PSE) as in (2.32).

$$\hat{P}(w_i) = \frac{1}{L}|X(w_i)|^2, \tag{2.30}$$

$$\hat{P}(w_i) = \frac{\hat{P}(w_i)}{\sum_{i=1}^{L} \hat{P}(w_i)}, \tag{2.31}$$

$$\hat{P}(w_i) = -\sum_{i=1}^{L} p_i \log p_i. \tag{2.32}$$

The work in [19] was able to bypass complexities such as extensive signal processing and latency by extracting some range-based statistical features from the 1D range FFT plot, such as the **height**, **width** and **area under peaks**, as well as **peak standard deviation**.

The **magnitude based features** presented below proved to be helpful in the detection of six specific hand movements using a CMOS Radar, as described in [20]. They can be derived after applying a Two-Dimensional Fast Fourier Transform (2D-FFT) to the signals received from the ADC component, recurring to a Range Doppler Image (RDI) based on the magnitude of the complex 2D FFT, with Z_i, R_i and D_i corresponding to the magnitude, range and doppler values of each index i from RDI image [20]. They are described as

Weighted Range

$$U = \frac{\sum_i Z_i.R_i}{\sum_i Z_i}, \tag{2.33}$$

Weighted Doppler

$$V = \frac{\sum_i Z_i.D_i}{\sum_i Z_i}, \tag{2.34}$$

Instantaneous Energy

$$I = \sum_i Z_i, \tag{2.35}$$

Range Dispersion

$$RD = \sqrt{\frac{\sum iZ_i.(R_i - U)^2}{\sum iZ_i}}, \tag{2.36}$$

Doppler Dispersion

$$DD = \sqrt{\frac{\sum iZ_i.(R_i - V)^2}{\sum iZ_i}}. \tag{2.37}$$

After developing a Dynamic range-Doppler trajectory [21] extracted various features that could aid in identifying six different human motions including falling, walking, and stepping, among others. Six of them are listed as follows

Dynamic Doppler Frequency is a sequence of Doppler values along a trajectory over time.

$$D(p) = k_p, \tag{2.38}$$

Dynamic Range Change is a progression of range coordinates over time.

$$\Delta R(i) = m_{i+1} - m_i, \tag{2.39}$$

Dynamic Energy Change is a progression of energy values over time.

$$\Delta E(i) = E_{i+1} - E_i, \tag{2.40}$$

Dynamic Dispersion of Range and Doppler are computed given the SD of range and doppler coordinates of the relevant data points.

$$Dis_D(p) = STD(k_{pq}), \tag{2.41}$$

$$Dis_R(p) = STD(m_{pq}). \tag{2.42}$$

2.3.3 Feature Engineering

The goal of feature engineering is to enhance the accuracy of machine learning models by choosing an appropriate feature set. This can be accomplished by gathering features from raw data and choosing the most relevant ones while taking into consideration both the particular problem setting and data characteristics.

2.3.3.1 Feature Selection

The ability to appropriately identify important features of data is essential in practical situations, especially when dealing with high-dimensional data, [22]. It can be performed in different ways, the main difference being the criteria used to determine the relevance of a feature and the integration of the selection process within the machine learning workflow. The three types of methods are listed below:

Filter Methods Filter methods are used to determine whether a feature is relevant regardless of the learning algorithm used. This process is typically performed by ranking the features based on a specific criterion and constructing a subset containing the highest-ranking features. Subsequently, only the pre-selected features are utilized in the classification algorithms.
An example of a filter method based on Correlation Criteria is Pearson's Correlation Coefficient. On the other hand, based on Mutual Information Criteria, namely

Shannon's definition of Entropy and the Kullback–Leibler divergence [23]. Overall, Filter Methods are simple to implement and computationally efficient since they do not involve the training of the machine learning model.

Wrapper Methods These methods work by evaluating the performance of a machine learning model with a subset of features and using that evaluation to determine the relevance of those features. Therefore, feature selection is dependent on the learning model used [24]. The process is typically iterative, and for that reason, wrapper methods can be more computationally expensive than filter methods when working with many features [25] but have been shown to obtain better performance due to being evaluated using a real modelling algorithm [26]. This advantage also suggests that the chosen subset of features may not produce the same outcome when applied to other learning models, since it has been specifically optimized for that model. Given their iterative nature, the search for which subsets to be tested can be conducted in two different ways, according to **Sequential Selection Algorithms** or **Heuristic Search Algorithms** [23].

Embedded Methods As the name implies, these methods are intrinsic to the learning algorithms, involving feature selection in the training process, which proves to reduce computation time. [23]. Embedded methods may combine filter and wrapper methods so that a pre-selection of features is performed before training, and the best subset is chosen among those, respectively. Some common examples of embedded feature selection methods include the **LASSO Regression** [26] and minimum Redundancy Maximum Relevance criterion (mRMR) [23].

2.3.3.2 Feature Extraction

Feature extraction creates new features from an original dataset not only to minimize processing resources but also to maintain relevant information. This process intends to tackle the dimensionality problem and involves a transformation of the original features through algebraic methods and optimization criteria. Additionally, feature extraction is less prone to overfitting and generally results in good classification accuracy compared to feature selection methods [27].

There are several feature extraction techniques available today, namely the popular Principal Component Analysis (PCA) technique and others such as Latent Semantic Indexing (LSI), Isometric Mapping (ISOMAP) and Linear Discriminant Analysis (LDA).

Principal Component Analysis (PCA) As previously stated, PCA is a feature extraction technique, which indicates that it is used for dimension reduction. It works by identifying the directions (principal components) in the data that explain the most variance and projecting the data onto a lower-dimensional space defined by these directions, through an orthogonal transformation of the input features. According

to [28], the number of principal components used is of great importance, and it can be chosen based on criteria such as cross-validation, and cumulative percentage of variance, among others.

Overall, PCA aims to find the most informative representation of data and can be used in image and signal processing, machine learning and statistics. Several articles have stated that PCA is a powerful tool, surpassing other alternatives such as LSI [28]. The authors of [27] mention that it is one of "the ideal feature extraction based dimensionality reduction" methods, among ISOMAP and LDA. The paper also analyzed several works which did experiments with many extraction techniques and drew the conclusion that the best accuracy was achieved when the PCA algorithm was optimized through deep learning, with Convolutional Neural Networks (CNNs).

As a result, we will only cover an analysis of the PCA technique.

It is possible to break down PCA into 5 simple steps:

1. First, it is necessary to transform the data to comparable scales in order to prevent biased results, since PCA can be sensitive to the variable's variances. This is done through the normalization of the data. Given n total features, and assuming each one ranges between m values, it is possible to build a nxm matrix. For the k-th row 1xm of features, its normalization X_n^k can be described as
$$X_n^k = \frac{X_k - \mu_k}{\sigma_k}, \tag{2.43}$$
with X^n being the k-th feature values vector.
2. Next, the covariance matrix C is obtained, which will reflect if there is a relationship between the input features. It is a nxn matrix and its cells are the covariance between all the possible pairs of features.
3. To identify the principal components it is necessary to decompose matrix C into a canonical form, whereby it is represented in terms of its eigenvalues and eigenvectors. The eigendecomposition can be described by
$$CV_k = \lambda_k V_k, \tag{2.44}$$
where V_k and λ_k are the eigenvector and eigenvalue of the k-th feature respectively. These new variables, principal components, are uncorrelated and most of the information present in the original variables is retained in the first components, which are the eigenvectors with the highest eigenvalue.
4. It is now possible to define the original data set along the direction of the principal components. Each eigenvector represents a direction, and its respective eigenvalue represents the magnitude of that direction. The data can be projected in a new space by the following expression
$$X_u = VX. \tag{2.45}$$

5. If the goal is dimension reduction, this step is fundamental, since it is possible to choose p principal components out of the total n existent. The key is to exclude the components with less eigenvalue, which by default have less information, and a new features vector, X_p, is defined with the remaining ones as

$$X_{up} = V_p X, \tag{2.46}$$

where V is the qxq matrix composed of the chosen eigenvectors, that better represent the feature set.

Each eigenvalue corresponds to the variance of its respective eigenvector. Thus, the total variance of the new feature set can be defined as the sum of all eigenvalues. As stated before, it is possible to reduce dimension by decreasing the number of principal components from n to p, which then affects the total variance. This change can be quantified by the variance ratio described as

$$r_{var} = \frac{\sum_{i=1}^{p} \lambda_i}{\sum_{j=1}^{n} \lambda_j}. \tag{2.47}$$

2.4 Machine Learning

2.4.1 Introduction to Machine Learning Techniques

Artificial Intelligence (AI) refers to the process of implementing in machines the capability to independently solve problems and perform tasks based on their own intelligence. Advanced algorithms and techniques make it possible for a machine to mirror human cognitive capabilities.

ML is the branch of AI that encompasses algorithms and statistical models that enable computers to acquire knowledge without being explicitly programmed to. The origins of machine learning actually trace back to 1959 relating to the game of checkers, when Arthur Samuel studied some *learning* methods that could be used to make a computer play the game [29].

ML today is subdivided into three learning categories: supervised, unsupervised and reinforcement learning. The first is the most common and relies on the concept of labels to learn the outputs of a system and identify data patterns to predict results [30]. Conversely, instead of working with labelled data, unsupervised learning uses input data alone to discover patterns and relationships within it. Lastly, in reinforcement learning, the algorithm learns by interacting with its environment through trial and error, analyzing the rewards or penalties associated with different actions and ultimately preferring the strategy of decisions that maximizes the rewards [30].

Several works have utilized ML to develop FMCW radar systems for human motion recognition, and the following paragraphs introduce the most commonly used techniques for that purpose. It is noteworthy to state that this thesis dataset is labelled, which leads

to a supervised learning approach.

K-Nearest Neighbor (KNN) is a simple supervised learning algorithm commonly used for ML, more specifically, to predict results in the stage of classification. It uses a labelled dataset of several data points from various classes to determine the class for a new data point. The key idea behind this technique is that the nearby K data points, and the classes they belong to, impact the class of this new point. The value K plays an important role and influences how effective this algorithm can be [31]. K-Nearest Neighbour (KNN) can be expensive to implement due to high memory requirements and computational complexity for larger datasets. This is because it is harder to determine an adequate value of K, since it's necessary to store all the training data and calculate the distances between the new point and the stored data points, making the prediction phase slower than other algorithms, [31].

Naive Bayes establishes a classification that is based on Bayes' Theorem. It predicts results derived from the probabilities of each class, as well as the conditional probability of belonging to a class, given a feature [32]. It all works with the presumption that input variables are independent of one another, hence the term naive, meaning that the value of a feature doesn't relate to the value of the remaining ones. Therefore, to achieve accurate predictions, input variables must not have a significant correlation, which isn't always true for some real-life datasets. This solution has been shown to be scalable and relatively easy to implement with adequate datasets, and it can be trained efficiently.

Support Vector Machine (SVM) is a supervised learning model that is based on a linear classification technique. It uses a hyperplane to distinguish classes, and its dimension is based on the number of inputs in the dataset, as well as a weight vector and a bias factor [33]. In order to divide classes accurately, it is desirable for the hyperplane to be distanced as far as possible from the support vectors, which are the data points closest to the hyperplane. Although the algorithm provides high accuracy and robustness, even for a large number of classes, the computational complexity during training time can be quite high [34]. In addition, some datasets may not be linearly separable. That explains the introduction of non-linear functions, Kernel functions, to improve data classification. The idea behind using them with Support Vector Machines (SVMs) is to bring the data into a higher-dimensional space, where it may be easier to find a linear separation between the different classes, thus defining an optimal hyperplane.

Artificial Neural Networks (ANNs) are inspired by some facets of biological neural networks [35], and consequently can excel at predicting results and identifying data patterns even for vague and fuzzy datasets, such as real-world most common ones. This is due to their interconnected "neurons" architecture: there is an input and an output layer, and between the two there can be several other layers, each of them constituted by one or more neurons that apply a linear operation to the input data they receive. Since they're

interconnected, the output will be the input of the next layer, and so forth. Despite being flexible, to effectively learn from datasets, Artificial Neural Networks (ANNs) require large amounts of data and computational power.

2.4.2 Deep Learning

Within ML, it is possible to define a subfield called Deep Learning (DL), which, in summary, refers to the use of neural networks with multiple hidden layers, [30]. In this section, an overview of ANNs is presented, as well as some of its variants such as CNNs, Recurrent Neural Networks (RNNs), Generative Adversarial Networks (GANs) and finally Autoencoders (AEs).

Deep Learning has become increasingly popular mainly due to its natural capacity to represent complex patterns in data, enabling tasks such as image recognition, natural language processing, and game playing to be performed at a state-of-the-art level.

2.4.2.1 Artificial Neural Networks

This subsection is divided in three parts each containing important information related to the functioning principle of ANNs.

1. Artificial Neurons

Mimicking the human body, ANNs' architecture is composed of artificial neurons. Also denoted as perceptrons, these building blocks are capable of receiving data as input, processing it and transmitting the output to other neurons, which will receive it as their input. The resemblance to human structures is clear since biological neurons are composed of dendrites which also receive signals from other neurons; a nucleus and an axon which feeds the signal to other neurons.

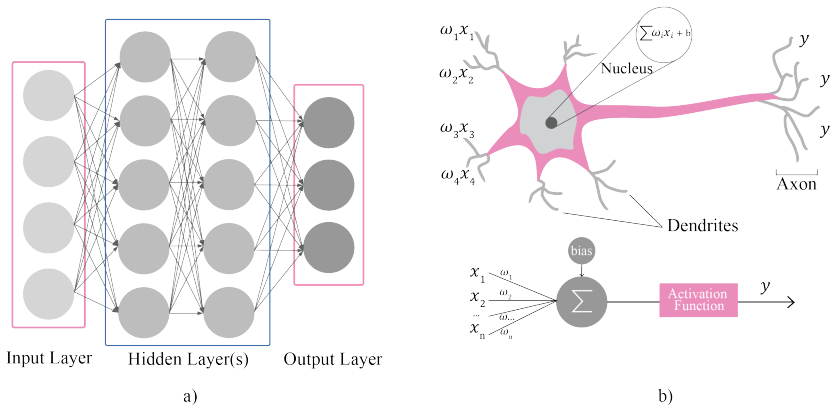

Figure 2.6: a) Connected Neural Network b) Analogy between biological and artificial neurons.

As depicted in Figure 2.6, it is possible to understand that similarly to the biological structure, an artificial neuron processes n signals received from n different neurons. The

neuron then attributes weight to each of the input signals, representing their importance. The signals are also processed recurring to a bias factor, a scalar that is summed after the weight function. Lastly, the output is formed based on an activation function. A brief description of these functions is presented in the following paragraphs.

2. Activation Funtions

These functions are responsible for the output y of the neuron. As illustrated in Figure 2.6 , their input consists of the weighted sum of the signals and bias factor. As a result, these functions can be generally expressed as

$$y = f(wx + b). \tag{2.48}$$

Activation functions are key in the accuracy of an ANN as they directly impact the network's output. Their shape determines the labelling of data points in a dataset, making it crucial to choose the appropriate activation function for a specific dataset, as there is a wide variety available.

As a result, several activation functions have been developed over the years. The simplest one is the linear function in (2.48). However, as mentioned before, sometimes a simple linear function might not be enough to properly distinguish between classes in the dataset. Hence the search for non-linear functions, as the ones presented below.

Sigmoid: The sigmoid function is a popular non-linear activation function, defined as

$$\sigma(x) = \frac{1}{(1 + e^{-x})}, \tag{2.49}$$

that returns values between 0 and 1. Its plot has an S shape as depicted in Figure 2.7. The sigmoid function is not commonly used in deep neural networks due to the vanishing gradient problem [36], which hinders the network's training. This problem can cause gradients to become very small during backpropagation, which makes it difficult for the network to learn by updating its weights, leading to slow convergence or poor performance. It is caused by the sigmoid function's output being saturating at either 0 or 1, which means that the gradient is close to 0 in those regions. An alternative solution is to use activation functions such as Rectified Linear Units (ReLUs).

Hyperbolic Tangent: This function can be described in (2.50). However, it's also possible to define it based on the sigmoid function previously introduced, as described in (2.51).

$$tanh(x) = \frac{sinh(x)}{cosh(x)}, \tag{2.50}$$

$$tanh(x) = 2sigmoid(2x) - 1. \tag{2.51}$$

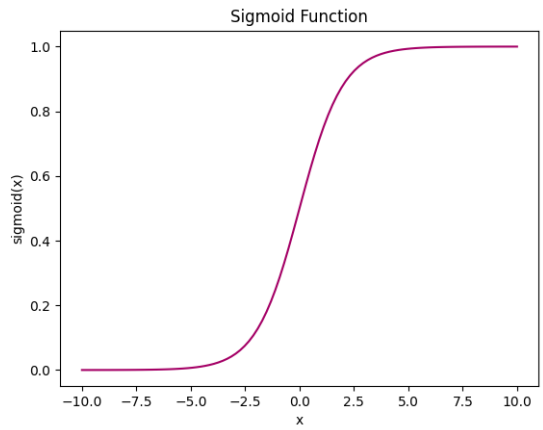

Figure 2.7: Sigmoid function plot.

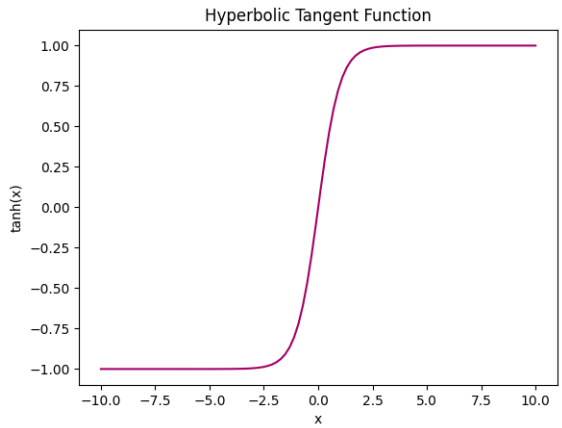

Figure 2.8: Hyperbolic Tangent function plot.

Comparing Figures 2.7 and 2.8, it is possible to see the resemblance between the hyperbolic tangent and the sigmoid functions. Apart from being centered in 0, the similarity makes *tanh* susceptible to the same vanishing gradient problem [36].

Rectified Linear Unit: ReLU is a popular approach for a non-linear activation function. It returns the input if it is positive, and returns 0 if it is negative. The mathematical formula for the ReLU function is expressed by

$$y(x) = max(0, x). \tag{2.52}$$

As opposed to the previous activation functions, ReLU helps alleviate the vanishing gradient problem in deep neural networks. It only has saturation issues in the positive region, making it useful for training the network's weights. The gradient of the ReLU function is always 1 for input values greater than 0. ReLU's plot is depicted in Figure 2.9.

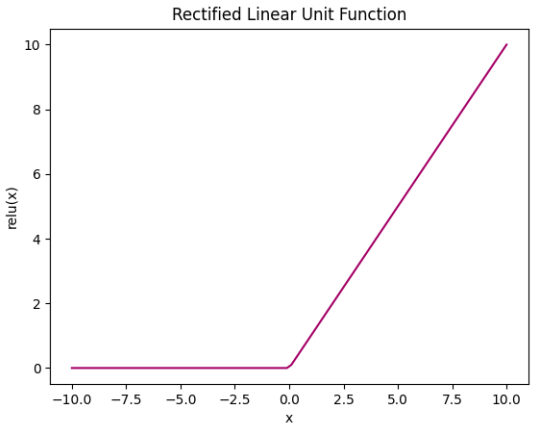

Figure 2.9: Rectified Linear Unit function plot.

Furthermore, ReLU are considered a breakthrough since they promise cheaper computation than hyperbolic tangent and sigmoid functions and can achieve good performance without relying on unsupervised pre-training [36]. However, they are still susceptible to the "dying ReLU" issue, related to the death of some neurons caused by "left-hard-saturation" [36].

There have been developed several adaptations of the ReLU such as LReLU, PReLU and RReLU, that can help overcome that issue and enhance the performance of the network. Instead of a null gradient in the negative region, Leaky Rectified Linear Unit (LReLU) has a small, non-zero value. Parametric Rectified Linear Unit (PReLU) introduces a learnable parameter that controls the slope of the negative part of the function. Randomized Rectified Linear Unit (RReLU) also introduces a learnable parameter but it is randomly sampled during training, rather than being learned.

Softmax: The softmax function is commonly used in the output layer of neural networks for classification tasks. It is a generalization of the logistic function that allows for more than two classes. Given an input vector of real numbers, the softmax function outputs a vector of K values, where each represents the probability of the input belonging to a particular class. By normalizing the output so that the sum of all the vector's values is 1, it ensures all the values are positive. The function is defined for each neuron i as

$$\sigma(x) = \frac{e^{x_i}}{\sum_k e_k^x}. \tag{2.53}$$

3. Training an ANN

Training an artificial neural network involves adjusting the network's parameters such as the weights and biases of its neurons to minimize the error between the network's predictions and the actual output (or label), for a given set of input data. It is possible to divide this process into the following stages. First, it is necessary to **split the dataset** into

subsets. The use of training, validation, and testing sets aids in evaluating the model's performance, avoiding overfitting, and fine-tuning the model.

Next, the **output of the network is calculated**. The output represents the prediction of the model for a given input, and by comparing that prediction to the validation subset, it's possible to **calculate prediction errors**. This can be accomplished with *loss* functions such as Mean Squared Error (MSE), Mean Absolute Error (MAE) and Cross-Entropy. To minimize the error, **weights are adjusted** for each neuron in the network through *backpropagation*, which involves propagating the error from the output layer to the input layer.

The output of the network is recalculated, and the described process repeats until either the error is minimized to the smallest value or until an established maximum of iterations is reached.

In summary, training is crucial for improving the accuracy of predictions and building a successful model. This includes avoiding issues such as overfitting and underfitting, which can lead to poor performance. *Overfitting* happens when a model has been optimized for the training data, but has difficulty generalizing to data that is different from the one with which it was trained. *Underfitting* however, reflects a model that is not complex enough to capture patterns in the data, resulting in poor performance on both training and testing data. Proper training can help mitigate these challenges.

2.4.2.2 Convolutional Neural Networks

A CNN is a type of deep learning neural network commonly used for image and video recognition tasks since they excel at finding more complex patterns in data. Convolutional, pooling, and fully connected layers constitute a CNN [37].

Convolutional layers can extract features from the input image by sliding a filter over it and computing element-wise multiplication between the entries of the filter and the image. These filters are also denoted as kernels and consist of mxn matrices. Each convolution results in a new matrix that represents the presence of certain features in the input data, a feature map. The process is repeated for each filter, creating multiple feature maps. The convolution operation is defined as:

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i-m, j-n)K(m,n), \tag{2.54}$$

where I represents the input image fed to the CNN, K is the kernel and (i,j) the coordinates of the convoluted image.

The connectivity pattern is different than in standard ANNs: according to [37], instead of having full connections between layers, kernel functions of reduced dimensions allow neurons in a certain layer to connect only to a specific, limited area of the layer before it. Because fully connected layers are made up of neurons that connect to every neuron in the layer before them, there are many parameters and a high level of computational complexity. Conversely, CNNs are more computationally efficient since neurons only have

to learn parameters from a small region of input data. A typical representation of a CNN is depicted in Figure 2.10.

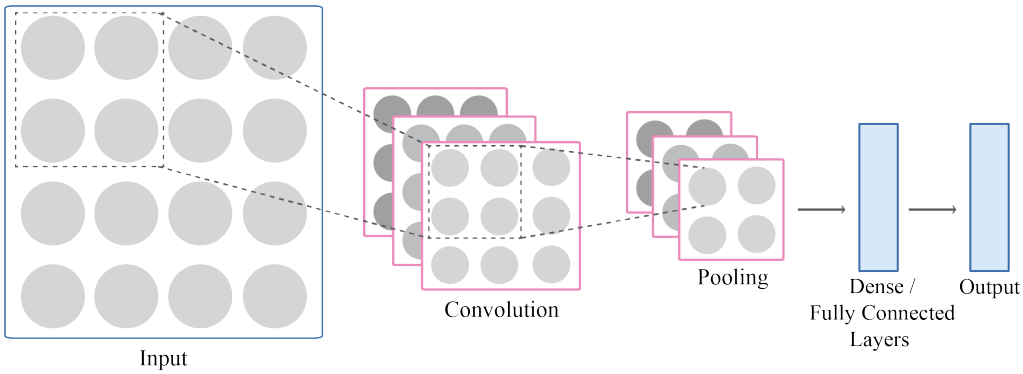

Figure 2.10: CNN architecture inspired by [38].

Pooling layers are then used to down-sample the feature maps, reducing the dimensionality of the data. They contribute to the model's increased resistance to minor distortions in the input image. This operation can be performed by selecting the maximum value of a region, *max pooling*, or by calculating the average of all values of a region, *average pooling*.

Lastly, fully connected layers or dense layers make predictions based on the processed data, similar to the case of standard ANNs.

2.4.2.3 Recurrent Neural Networks

RNNs are a type of neural network equipped with the capability of processing temporal information by allowing their neurons to exchange information through feedback loops. Thus, neurons can receive feedback from other neurons in the network [39].

RNNs can be unrolled to show a sort of "memory" through time, allowing previous information to persist in the network to help fine-tune further outputs, hence the name "recurrent". This can improve their robustness towards unknown data that differs from training data, reducing the chances of overfitting as well as underfitting.

These networks work especially well with sequential data, such as time series (i.e., a sequence of musical notes in [40]) or natural language, being able to predict the next character in a string [40], and being useful for tasks such as language translation and speech recognition. Most of these networks can even process sequences with variable lengths. Figure 2.11 depicts a typical RNN architecture.

Each RNN cell behaves according to Equations (2.55) - (2.57), where $h(t)$ represents the hidden state, which serves as the network's "memory", and $x(t)$ represents the cell's input at time instant t.

$$a^{(t)} = b + Wh^{(t-1)} + Ux^{t}, \tag{2.55}$$

$$h^{(t)} = g(a^{(t)}), \tag{2.56}$$

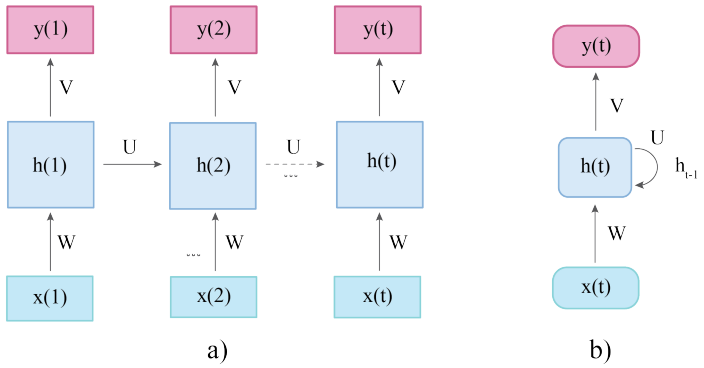

Figure 2.11: a) Unfolded RNN architecture b) Folded RNN inspired by [41].

$$o^{(t)} = c + Vh^{(t)}, \tag{2.57}$$

$$y^{(t)} = g(o^{(t)}), \tag{2.58}$$

The Equation (2.55) translates the aggregation of all inputs a cell receives at time step t, $a(t)$, which include the new input $x(t)$ and the previous hidden state $h(t-1)$, as well as the bias factor b of the own cell. W and U both represent weight matrices associated with the hidden-to-hidden and input-to-hidden connections. The hyperbolic tangent function is regularly used as an activation function for the output of each cell of the network, thus, in (2.56), the g function can be replaced with $tanh$. Equation (2.57) represents the network's output $o(t)$ at time step t, which is dependent on the bias factor c as well as the hidden state $h(t)$ at that moment. Generally, the output of the last layer in the network, the output layer, is submitted to one last activation function g, as represented in (2.58). It is common to use the softmax function for this purpose.

The most common type of RNNs are Long Short-Term Memorys (LSTMs) and Gated Recurrent Units (GRUs) since they overcome average RNN problems such as the vanishing gradient and the exploding gradient. The first one has been briefly explained in previous sections. However, exploding gradient refers to the case when the network's weights receive large updates, resulting in an unstable model that cannot learn from data, [42, 43].

Long Short-Term Memory - LSTM

LSTM is a variation of a RNN designed to handle long-term dependencies in sequential data. Aiming to overcome the vanishing gradient problem that occurs in traditional RNNs, a special type of memory cell and gates that control the flow of information were created. These allow LSTMs to better maintain information from previous time steps, which makes them ideal for tasks such as language modelling, speech recognition, and time series forecasting. Figure 2.12 depicts a typical LSTM architecture.

LSTMs introduce a new concept of cell state, $c(t)$, that is updated at each timestep by a combination of the previous state, the current input, and the output of three different gates (input gate, $i(t)$, forget gate, $f(t)$, and output gate, $o(t)$) in LSTM cell. These gates

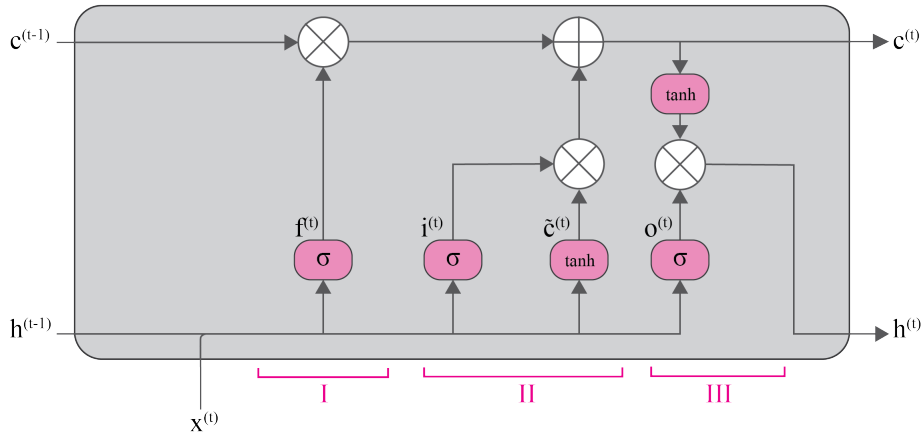

Figure 2.12: LSTM architecture (adapted from [41]).

control the flow of information into and out of the state vector and allow the LSTM to selectively forget or preserve information as it processes the sequence. The following paragraphs clarify each of these gates' functions.

I. The **forget gate** constitutes a $sigmoid$ neuron that takes as input the current input $x^{(t)}$ and the previous state $h^{(t-1)}$ and outputs a value between 0 and 1, as described by

$$f^t = \sigma(W_f \cdot [x^{(t)}, h^{(t-1)}] + b_f), \tag{2.59}$$

where W_f and b_f are weight and biases parameters, respectively. This value is multiplied by the previous state vector, which is scaled by a value between 0 and 1, effectively "forgetting" some of the previous states. This gate plays an important role in allowing the LSTM to focus on the most relevant information and ignore irrelevant or outdated information.

II. The **input gate** is responsible for controlling the flow of information into the cell's state vector. It is a $sigmoid$ neuron that also receives the current input and the previous state, and outputs a value between 0 and 1. Its operation is described by

$$i^t = \sigma(W_i \cdot [x^{(t)}, h^{(t-1)}] + b_i), \tag{2.60}$$

where W_i and b_i are weight and biases parameters, respectively. This output will be multiplied by possible candidate values, $c(t)$, which result from the following expression

$$\tilde{c}^t = \tanh(W_c \cdot [x^{(t)}, h^{(t-1)}] + b_c), \tag{2.61}$$

where W_c and b_c are weight and biases parameters. Lastly, the result of this operation is summed with the result of the forget gate, resulting in the **cell state** for the current time step. It can be characterized by the following expression

$$c^t = f^{(t)} * c^{(t-1)} + i^{(t)} * \tilde{c}^t, \tag{2.62}$$

where $*$ denotes the Hadamard product.

III. The **output gate** influences what information is shared from the cell state to the hidden state, and consists of a $sigmoid$ neuron which also takes the current input and the previous state as input and outputs a value between -1 and 1, as described by the following expression

$$o^{(t)} = \sigma(W_o \cdot [x^{(t)}, h^{(t-1)}] + b_o), \tag{2.63}$$

where W_o and b_o are the weight and biases parameters, respectively.

The result of this operation it then multiplied with the output oh the hyperbolic tangent applied to the current cell state, $c(t)$, resulting in the **hidden state** as described by the following equation:

$$h^{(t)} = o^{(t)} * \tanh(c^{(t)}). \tag{2.64}$$

Gated Recurrent Unit - GRU

GRUs were first introduced in [44], were the first to describe , with the goal of eliminating the vanishing gradient problem. GRUs are one of many variations on traditional LSTMs that differ from the original by combining the cell state and hidden state, as well as the forget and input gates into a single update gate. GRU's architecture is depicted in Figure 2.13, where the only gates are the reset and update gates. The **reset gate** can select what

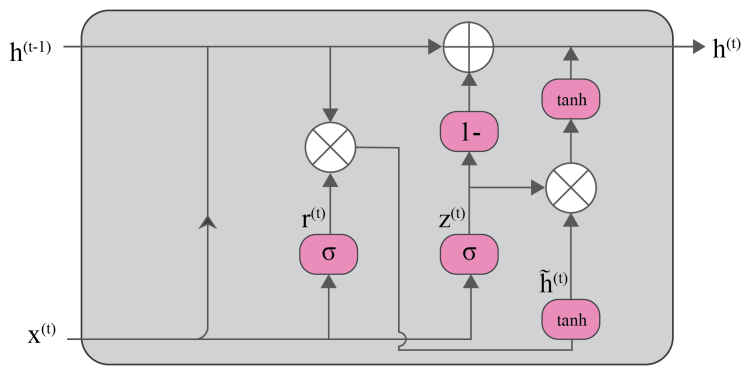

Figure 2.13: GRU architecture (adapted from [41]).

information to discard from the previous hidden state, as described by

$$r^{(t)} = \sigma(W_r \cdot [h^{(t-1)}, x^{(t)}] + b_r), \tag{2.65}$$

where W_r and b_r are weight and biases parameters, $h^{(t-1)}$ is the hidden state of the previous time step and $x^{(t)}$ is the current input vector. The other gate, **update gate** $z^{(t)}$, has the contrary function: it selects which information gets to flow into the hidden state vector. Its operation is described as

$$z^{(t)} = \sigma(W_z \cdot [h^{(t-1)}, x^{(t)}] + b_z), \tag{2.66}$$

where W_z and b_z are weight and biases parameters. A vector containing a list of possible candidates to add to the hidden state is given by

$$\tilde{h}^{(t)} = \tanh(W_h \cdot [h^{(t-1)}, x^{(t)}] + b_h), \tag{2.67}$$

where W_h and b_h are the learnt weights and biases. Finally, the current hidden state can be calculated based on the following expression

$$h^{(t)} = (1 - z^{(t)}) * h^{(t-1)} + z^{(t)} * \tilde{h}^{(t)}, \tag{2.68}$$

where $*$ denotes the Hadamard product.

In conclusion, while the architecture of GRUs is significantly simpler than that of LSTMs, their performance can be quite similar [44]. GRUs may be preferred due to their faster computation time, whereas the latter may be preferred due to slightly higher accuracy.

2.4.2.4 Generative Adversarial Networks

A GAN is a neural network architecture composed of two components: a generator network and a discriminator network. The generator creates new data samples synthesized from the learned distribution of training data, hence the name "*generative*", whereas the discriminator network attempts to tell generated and real samples apart. The two networks are trained in an adversarial process in which the generator attempts to build samples that will deceive the discriminator, which attempts to correctly identify the generated samples. A simple analogy is illustrated in [45], where these two entities are compared to an "*art forger* and an *art expert*". A typical GAN architecture is depicted in Figure 2.14.

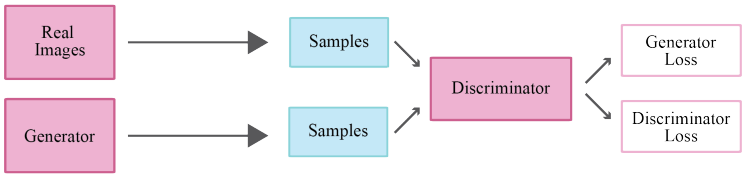

Figure 2.14: GAN architecture (adapted from inspired by [46]).

Training a GAN entails two distinct training processes. Training the discriminator is similar to training a simple classifier, which uses discriminator loss to penalize the discriminator each time it classifies a sample incorrectly. Both network's weights are updated using backpropagation based on the discriminator loss and generator loss. To train the generator, random noise is fed into it and transformed into a data instance. The discriminator then tries to sort the data, deciding if it is true or false, and the generator's loss is calculated using the discriminator's output. It is worthy to note that these two processes are alternatively repeated.

GANs are used for a wide range of tasks, including image synthesis, image-to-image translation and image superresolution [45]. However, they still suffer from a common

problem across deep learning models, the vanishing gradient problem: if the discriminator executes its job perfectly, the generator's training may fail. The mode collapse issue can also occur when a generator produces a limited set of outputs that are always accepted by the discriminator, causing both the generator and discriminator to get stuck in a local minimum, leading to reduced diversity in generated outputs. Failure to converge can also happen in GANs if the generator excels at creating perfect samples, leading the discriminator to fail at distinguishing between real or fake samples. There are ways to remedy those issues, as discussed in [46].

2.4.2.5 Auto Encoders

An autoencoder is a neural network in the field of unsupervised learning that is composed of an encoder and a decoder. The first learns to map input data into an internal and compact representation, and then the latter decodes it as accurately as possible, forming an output similar to the original data. The encoder and decoder are trained so that the discrepancy between the original input and the reconstructed output is reduced. With autoencoders it is possible to have outputs of the same dimension as inputs, their architecture is depicted in Figure 2.15.

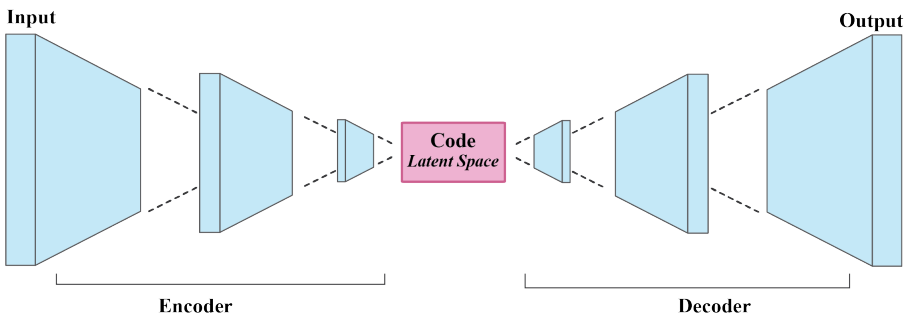

Figure 2.15: Autoencoder architecture.

The encoder generates code that gathers important features from input data, hence AEs can be compared to a generalization of the PCA algorithm, [47], also being useful for feature extraction approaches. Some AEs can be used as generative models, while others can be used to improve classification by performing feature extraction. AEs can also be used for anomaly detection and dimensionality reduction, [47].

2.5 State of the Art / Related Works

Some work has already been done in the area of RF sensing, namely for the purpose of human motion detection. A brief summary of some of this work will be presented in the next paragraphs, for a better understanding of the developments made so far, as well as to identify open challenges in RF sensing. The study of several works available in the literature also proved to be important to recognize the performance of some technologies

mentioned throughout the sections of this chapter when applied to the human motion detection context.

In [48], S. J. Ryu et al. introduce feature-based gesture recognition in a FMCW radar system. A Range-Doppler Map (RDM) is obtained from raw signals of FMCW radar and a variety of features are generated from it. The Quantum-Inspired Evolutionary Algorithm (QEA), a wrapper-based algorithm, was employed for feature selection, along with an *Information Factor* based on the mRMR. This combination, Quantum-Inspired Evolutionary Algorithm - Information Factor (QEA-IF), proved to be helpful in recognizing feature subsets related to gesture recognition, which were more useful and generally improve the accuracy of the predictions up to 85.29%. It is worth noting that the authors identified that the shape of the RDM map had little to no effect on gesture recognition and that intensity information was more useful for their implementation than range and velocity information. Therefore, the intensity of the most reflective point of the signal, ρ^*, and the total energy of received signals, E_{inst}, were identified as relevant features. Furthermore, a comparison between the performance of QEA-IF and other classifiers such as *KNN* and *Random Forest* also concluded that QEA-IF achieves better accuracy and faster recognition rate.

A research study on gesture recognition using a 77 GHz FMCW radar system was conducted by Y. Sun et al, [49], where the system utilizes micro-Doppler (μD) signatures to recognize driver's gestures for an in-vehicle driver monitoring application. Because these signatures were frequently distorted by the presence of multiple moving targets, and the goal was to recognize only the driver's hand gestures, range information was included to filter out the influence of irrelevant moving targets. The authors proposed a method using five handcrafted features and a KNN classifier, achieving an average prediction accuracy of 83.52%. However, for three of the gestures, the accuracy was 90%, while for the remaining two, which required more axial movement, the accuracy was only 63%. This is likely due to the radar's sensitivity to radial movements and lack of information on axial movements. The authors also discussed potential future research to address issues with wideband background noise and to implement feature selection methods such as PCA to improve training efficiency.

In [20], P. Goswami et al. describe a low-complexity multi-gesture classification solution for consumer and automotive electronics that employs a 77 GHz mm-wave low-power CMOS radar, as well as an innovative algorithm and software pipeline. It enables real-time gesture detection and uses an ANN for classification, using magnitude and phase-based features, as well as statistical features. Overall, 96% accuracy for 6 gestures was achieved across 8 users, which is an interesting result for a single-chip radar solution with limited memory, computation power, and angle resolution constraints.

In [21], C. Ding et al. proposed a method for recognizing continuous human motion using radar technology, specifically a FMCW radar system. The method, called Dynamic Range-Doppler Trajectory (DRDT), extracts relevant information from range-Doppler frames obtained from the radar's signals. Then, using a peak search method, each human

motion is located and separated, and multidomain features are extracted. These time, range, Doppler and RCS (radar cross section) features are fed to a KNN classifier, resulting in robust recognition accuracy of around 95% for single motions and 91.9% for continuous motions. Although the model performed well across various angles, individuals, and different distances and directions, allowing real-time recognition of human motion in real-living environments, the authors observed a slight deterioration in performance beyond a 30° view angle.

The work in [50] presents results for a FMCW radar that uses micro-Doppler signatures for fall detection. The researchers extract micro-Doppler signatures from radar signals and use transfer learning (*AlexNet*), SVM,KNN, and deep learning (*GoogleNet*) algorithms to automatically extract features and perform classification, respectively. The experiment involves several people performing six different activities in four different locations. The central idea is to create a generalized fall detection system that works regardless of location or subjects involved, which entails investigating the portability and applicability of radar datasets for Human Activity Recognition (HAR). All three classifiers produced significant results, with an average test percentage accuracy of 78.25% for SVM, 77.15% for KNN, and 74.7% for GoogleNet. Researchers stated that further developments could be pursued by taking more measurements in different locations and involving elderly people, with the aim of improving the test accuracy.

The use of RADAR sensors in short-range human monitoring applications, such as activity classification in smart homes and gesture recognition for human-computer interaction is discussed in [3]. Similar to other works, micro-Doppler signatures and range-time information are extracted from radar signals. For classification, a Bidirectional Long Short-Term Memory (Bi-LSTM) network is used, and unlike most related works that use CNNs, it interprets radar data as a temporal series rather than images. The ability to analyze these sequences, according to the authors, is one of their main contributions, allowing for more realistic data acquisition. Furthermore, they proposed a stacked Bi-LSTM system that is better suited to the temporal sequence nature of the data and achieves over 90% accuracy for 45 different activity sequences tested.

J. Bhatia et al., [19], adopted an FMCW radar to build a detection system that can distinguish between drones, humans, and cars. A range FFT plot is created based on the radar signals, and the authors claim that this is the first project relying on range FFT features to perform target classification. Two lightweight classification models, such as Logistic Regression and Naive Bayes, are explored to assess the performance. The selected features fed to the ML algorithms were based on the peaks detected in those plots, some of which included width, area, and standard deviation of peak values. Overall, Logistic Regression and Naive Bayes algorithms reached 86.9% and 73.9% accuracy, respectively, and the researchers note that some future work could be done to improve target size and shape detection and that range-Doppler plots could be introduced for more reliable classification. It should be noted that this paper contributes to advancements in efficient and dependable management of ground station traffic at a cost-effective price point as

well as computer vision and object recognition.

In [51], K. Han et al. propose a phase-extraction technique for a FMCW radar that allows the detection of target motions. The technique involves decomposing an FMCW chirp signal into multiple-frequency signals, 250 to be exact, to extract modulated phases due to target motions. This novel method can extract the phase without performing a FFT, which can overcome the range migration problem that causes errors in phase extraction using a conventional FFT-based method. It reduces noise and can improve detection accuracy for both human vital signals, which consist of small vibrations, and several body motions. Thus, the detection of phase can be precise regardless of the type of moving targets.

R. Cruz et al. [52] studied the importance of various features in classifying human mobility scenarios using an RF sensing system in the 76-81 GHz band. The authors compared the performance of different features computed from the RF time of flight and use four different methodologies to evaluate their importance: Mutual Information (MI), Analysis of Variance (ANOVA), the feature weighting algorithm *ReliefF*, and Recursive Feature Selection (RFE). Overall, 126 raw features were submitted to each feature selection method and glsKNN was used as a classifier. Despite all methodologies being capable of identifying features of lower importance, it was concluded that RFE leads to better accuracy in classification when its 18 selected features are used for classification. ANOVA demonstrated the lowest accuracy rate, and even PCA could not surpass RFE when the number of selected features was higher than 12.

3 Work Plan

This thesis combines, among other topics, two important fields of study: radar technology and machine learning.

It is therefore structured into several tasks that are designed to meet its objective. This chapter outlines the strategy, timeline, and steps involved, which helps to organize the research activities, allocate resources and maintain focus on the goal of the project. An overview of that goal was defined in Section 1.2, and in the following sections several tasks and milestones are defined to be achieved during the course of the study.

3.1 Task Description

The work plan for this thesis includes the following tasks:

1. **Overview of related work**

   This task includes the study and comprehension of Frequency Modulation Continuous Wave active radio frequency sensing. In a further step, the task includes the identification of popular machine-learning techniques adopted in the literature to address the topic of human posture and motion classification, as well as the type of features involved in the classification process and the signal processing methods to compute them

2. **Real-time Data Visualization and Features**

   This task involves the experimentation of different visualization tools capable of showing the dynamic behaviour of different features computed in real-time. The goal is to obtain a visual assessment of hypothetical features to adopt, how they change over time, and validate their adequacy for different types of motion.

3. **Identification and Characterization of Different features**

In this task, several datasets will be obtained to identify potential features that better represent the data's information. The analysis might include an initial characterization of the data's statistical properties and in a subsequent step, the identification and assessment of various types of features adopted in the classification process.

4. **Classification**

   This task handles the development of classification methodologies to identify the sensed context with high accuracy and in a short period of time. The implementation of a demonstration prototype is a secondary goal, once the most appropriate classification methodology has been identified.

5. **Performance Evaluation**

   This task evaluates the performance of the solutions proposed in tasks T1 to T4. During T1 the evaluation of related works takes place. The identified features and classification methods explored in tasks T2 and T3 will be compared by assessing the performance of each combination in real-time operation.

6. **Thesis Writing**

   This task includes the reporting activities in initial drafts to describe the proposed solutions and the performance results achieved in the first step. The documents will be updated over time to contain all the steps developed during the working period and, at the final stage, they include all the material required for each chapter of the MSc dissertation. This task can might also include the reporting of the achievements in a scientific paper.

3.2 Schedule

A Gantt chart with the time schedule for this thesis is presented in figure 3.1. It provides a visual representation of the thesis schedule that displays the start and finish dates predicted for the tasks.

The practical work will begin in February with the second task. The graph also shows that thesis writing began in January and will continue until September, owing to the fact that this report already contains information for some chapters of the thesis. Although the writing task is indicated in the chart as concluding in September, it should be noted that it will continue until all issues related to the growth of this thesis have been described. Although September 2023 is the stated deadline, it would be preferable and expected for the task to be completed sooner. Since it is possible to infer the performance of other studies dissected during T1 as well as the performance of the solution suggested in this thesis constructed over Tasks 2 and 3, the performance evaluation (T4) task runs concurrently with Tasks T1 to T3.

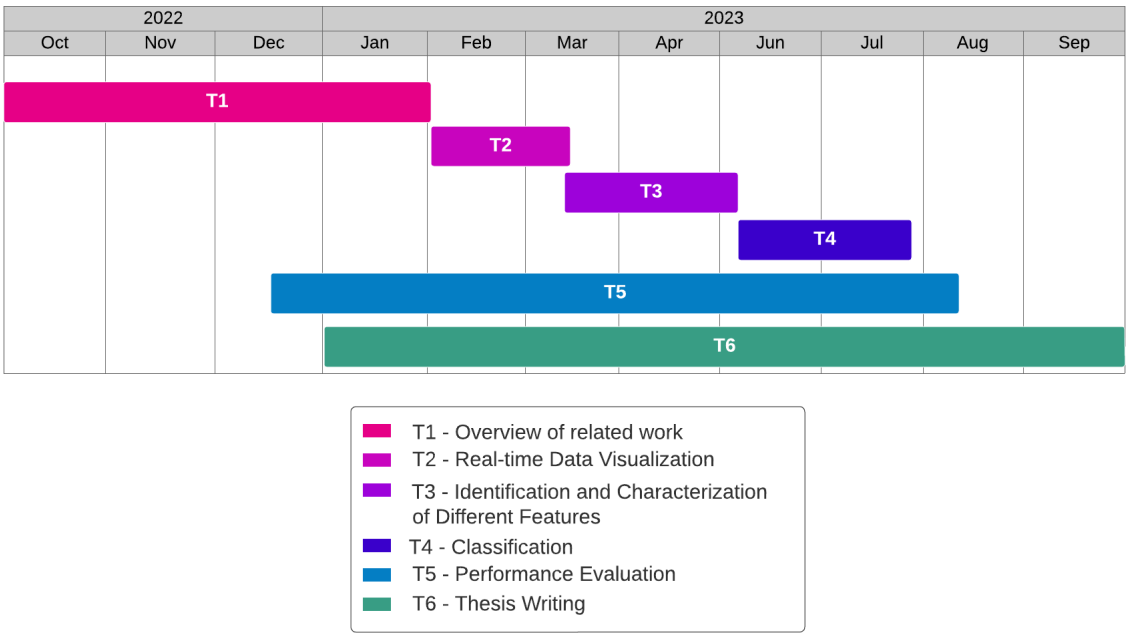

Figure 3.1: Gantt Chart for the timeframe of this thesis work plan.

4 Conclusions

In this report, it was possible to contextualize the scenario for the proposed thesis "Human Posture and Motion Classification through RF Signals", through a brief study of some modern technologies introduced in Chapter **??**.

In Chapter 2 it was possible to go into further detail about some of the technologies envisioned for the work to be pursued in this thesis. The fundamentals of FMCW operation were introduced, which brought to light the kind of technical gear that will be dealt with during the scope of this thesis. It was possible to understand how to gather range and velocity information, which is essential for classifying human motion.

From the multiple works already available in the literature, it was rapidly concluded that features played an important role in the behaviour of learning models, which led to a quick analysis of some of the possible candidates to implement for this thesis scenario, conducted in Chapter 2. Given the vast range of machine-learning models available today, it was necessary to understand each category's work principle to select the most appropriate model for this thesis.

Since there's some related work already conducted using the FMCW radar for classification of motion, the state of the art was also presented in Section 2.5, where a summary of a selection of articles was introduced, which led to a better understanding of what future work could be developed.

Lastly, a proposed schedule was constructed in Chapter 3, segmenting the work of this thesis into six different subtasks, which will be the focus of the work to be developed in the next months.

Bibliography

[1] A. Pramod, H. S. Naicker, and A. K. Tyagi. "Machine Learning and Deep Learning: Open Issues and Future Research Directions for the Next 10 Years". In: Wiley, 2021-07, pp. 463–490. DOI: 10.1002/9781119785750.ch18 (cit. on p. 3).

[2] T. Verdonck et al. *Special issue on feature engineering editorial*. 2021. DOI: 10.1007/s10994-021-06042-2 (cit. on p. 3).

[3] A. Shrestha et al. "Continuous Human Activity Classification from FMCW Radar with Bi-LSTM Networks". In: *IEEE Sensors Journal* 20 (22 2020-11), pp. 13607–13619. ISSN: 15581748. DOI: 10.1109/JSEN.2020.3006386 (cit. on pp. 3, 4, 32).

[4] S.-J. Ryu et al. "Feature-Based Hand Gesture Recognition Using an FMCW Radar and its Temporal Feature Analysis". In: *IEEE Sensors Journal* 18.18 (2018), pp. 7593–7602. DOI: 10.1109/JSEN.2018.2859815 (cit. on p. 3).

[5] K. Grm et al. "Strengths and weaknesses of deep learning models for face recognition against image degradations". In: *IET Biometrics* 7.1 (2018), pp. 81–89. DOI: https://doi.org/10.1049/iet-bmt.2017.0083. eprint: https://ietresearch.onlinelibrary.wiley.com/doi/pdf/10.1049/iet-bmt.2017.0083. URL: https://ietresearch.onlinelibrary.wiley.com/doi/abs/10.1049/iet-bmt.2017.0083 (cit. on p. 4).

[6] A. U. Kulkarni et al. "RADAR based Object Detector using Ultrasonic Sensor". In: *2019 1st International Conference on Advances in Information Technology (ICAIT)*. 2019, pp. 204–209. DOI: 10.1109/ICAIT47043.2019.8987259 (cit. on p. 4).

[7] M. V. Paulet, A. Salceanu, and O. M. Neacsu. "Ultrasonic radar". In: *2016 International Conference and Exposition on Electrical and Power Engineering (EPE)*. 2016, pp. 551–554. DOI: 10.1109/ICEPE.2016.7781400 (cit. on p. 4).

[8] T. K. Sarkar and M. Salazar Palma. "A history of the evolution of RADAR". In: *2014 44th European Microwave Conference*. 2014, pp. 734–737. DOI: 10.1109/EuMC.2014.6986539 (cit. on p. 5).

[9] M. Jankiraman. *FMCW Radar Design*. Artech House, 2018 (cit. on p. 5).

[10] D. F. Fioranelli et al. "Radar sensing for healthcare". In: *Electronics Letters* 55 (2019-09), pp. 1022–1024. DOI: 10.1049/el.2019.2378. (Visited on 2022-02-16) (cit. on p. 5).

[11] G. Kouemou. *Radar Technology*. IntechOpen, 2010. ISBN: 9789533070292. URL: https://books.google.pt/books?id=aRKaDwAAQBAJ (cit. on p. 5).

[12] R. Knight. "Ground penetrating radar for environmental applications". In: *Annual Review of Earth and Planetary Sciences* 29.1 (2001), pp. 229–255 (cit. on p. 5).

[13] F. Bekkadal. "Novel radar technology and applications". In: *17th International Conference on Applied Electromagnetics and Communications, 2003. ICECom 2003.* 2003, pp. 6–12. DOI: 10.1109/ICECOM.2003.1290942 (cit. on p. 5).

[14] K. Ramasubramanian and T Instruments. "Using a complex-baseband architecture in FMCW radar systems". In: *Texas Instruments* (2017) (cit. on p. 8).

[15] G. Brooker. "Understanding millimetre wave FMCW radars". In: *1st International Conference on Sensing Technology* (2005-01) (cit. on pp. 8, 9).

[16] E. Guerrero-Menéndez. "Frequency-modulated continuous-wave radar in automotive applications". In: 2018 (cit. on pp. 10, 11).

[17] C. S. Lessard. "Window Functions and Spectral Leakage". In: *Signal Processing of Random Physiological Signals*. Cham: Springer International Publishing, 2006, pp. 175–193. ISBN: 978-3-031-01610-3. DOI: 10.1007/978-3-031-01610-3_16. URL: https://doi.org/10.1007/978-3-031-01610-3_16 (cit. on p. 13).

[18] *Understanding FFTs and Windowing*. www.ni.com, 2021-12. URL: https://www.ni.com/pt-pt/innovations/white-papers/06/understanding-ffts-and-windowing.html (visited on 2023-01-23) (cit. on p. 13).

[19] J. Bhatia et al. "Object Classification Technique for mmWave FMCW Radars using Range-FFT Features". In: *2021 International Conference on COMmunication Systems & NETworkS (COMSNETS)*. 2021, pp. 111–115. DOI: 10.1109/COMSNETS51098.2021.9352894 (cit. on pp. 14, 32).

[20] P. Goswami et al. "Real-Time Multi-Gesture Recognition using 77 GHz FMCW MIMO Single Chip Radar". In: *2019 IEEE International Conference on Consumer Electronics (ICCE)*. 2019, pp. 1–4. DOI: 10.1109/ICCE.2019.8662006 (cit. on pp. 14, 31).

[21] C. Ding et al. "Continuous Human Motion Recognition With a Dynamic Range-Doppler Trajectory Method Based on FMCW Radar". In: *IEEE Transactions on Geoscience and Remote Sensing* 57.9 (2019), pp. 6821–6831. DOI: 10.1109/TGRS.2019.2908758 (cit. on pp. 15, 31).

[22] V. Bolón-Canedo, N. Sánchez-Maroño, and A. Alonso-Betanzos. "A review of feature selection methods on synthetic data". In: *Knowledge and Information Systems* 34 (2012-03), pp. 483–519. DOI: 10.1007/s10115-012-0487-8 (cit. on p. 15).

[23] G. Chandrashekar and F. Sahin. "A survey on feature selection methods". In: *Computers & Electrical Engineering* 40.1 (2014). 40th-year commemorative issue, pp. 16–28. ISSN: 0045-7906. DOI: https://doi.org/10.1016/j.compeleceng.2013.11.024. URL: https://www.sciencedirect.com/science/article/pii/S0045790613003066 (cit. on p. 16).

[24] S. Maldonado and R. Weber. "A wrapper method for feature selection using Support Vector Machines". In: *Information Sciences* 179 (2009-06), pp. 2208–2217. DOI: 10.1016/j.ins.2009.02.014. (Visited on 2018-11-19) (cit. on p. 16).

[25] A. Wang et al. "Accelerating wrapper-based feature selection with K-nearest-neighbor". In: *Knowledge-Based Systems* 83 (2015), pp. 81–91. ISSN: 0950-7051. DOI: https://doi.org/10.1016/j.knosys.2015.03.009. URL: https://www.sciencedirect.com/science/article/pii/S0950705115001033 (cit. on p. 16).

[26] A. Jović, K. Brkić, and N. Bogunović. "A review of feature selection methods with applications". In: *2015 38th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*. 2015, pp. 1200–1205. DOI: 10.1109/MIPRO.2015.7160458 (cit. on p. 16).

[27] R. Zebari et al. "A comprehensive review of dimensionality reduction techniques for feature selection and feature extraction". In: *Journal of Applied Science and Technology Trends* 1.2 (2020), pp. 56–70 (cit. on pp. 16, 17).

[28] F. P. Shah and V. Patel. "A review on feature selection and feature extraction for text classification". In: *2016 International Conference on Wireless Communications, Signal Processing and Networking (WiSPNET)*. 2016, pp. 2264–2268. DOI: 10.1109/WiSPNET.2016.7566545 (cit. on p. 17).

[29] IBM. *What is machine learning?* www.ibm.com. URL: https://www.ibm.com/topics/machine-learning (cit. on p. 18).

[30] P. Ongsulee. "Artificial intelligence, machine learning and deep learning". In: *2017 15th International Conference on ICT and Knowledge Engineering (ICT&KE)*. 2017, pp. 1–6. DOI: 10.1109/ICTKE.2017.8259629 (cit. on pp. 18, 20).

[31] K. Taunk et al. "A Brief Review of Nearest Neighbor Algorithm for Learning and Classification". In: *2019 International Conference on Intelligent Computing and Control Systems (ICCS)*. 2019, pp. 1255–1260. DOI: 10.1109/ICCS45141.2019.9065747 (cit. on p. 19).

[32] S. K et al. "Career Prediction Using Naive Bayes". In: *2022 Third International Conference on Intelligent Computing Instrumentation and Control Technologies (ICICICT)*. 2022, pp. 695–699. DOI: 10.1109/ICICICT54557.2022.9917745 (cit. on p. 19).

[33] A. C. Braun, U. Weidner, and S. Hinz. "Support vector machines, import vector machines and relevance vector machines for hyperspectral classification — A comparison". In: *2011 3rd Workshop on Hyperspectral Image and Signal Processing: Evolution in Remote Sensing (WHISPERS)*. 2011, pp. 1–4. DOI: 10.1109/WHISPERS.2011.6080861 (cit. on p. 19).

[34] Y. Yang, J. Li, and Y. Yang. "The research of the fast SVM classifier method". In: *2015 12th International Computer Conference on Wavelet Active Media Technology and Information Processing (ICCWAMTIP)*. 2015, pp. 121–124. DOI: 10.1109/ICCWAMTIP.2015.7493959 (cit. on p. 19).

[35] M. Chen et al. "Artificial Neural Networks-Based Machine Learning for Wireless Networks: A Tutorial". In: *IEEE Communications Surveys & Tutorials* 21.4 (2019), pp. 3039–3071. DOI: 10.1109/COMST.2019.2926625 (cit. on p. 19).

[36] B. Ding, H. Qian, and J. Zhou. "Activation functions and their characteristics in deep neural networks". In: *2018 Chinese Control And Decision Conference (CCDC)*. 2018, pp. 1836–1841. DOI: 10.1109/CCDC.2018.8407425 (cit. on pp. 21–23).

[37] K. O'Shea and R. Nash. "An Introduction to Convolutional Neural Networks". In: *ArXiv e-prints* (2015-11) (cit. on p. 24).

[38] T. Wang et al. *Deep Learning for Wireless Physical Layer: Opportunities and Challenges*. 2017. DOI: 10.48550/ARXIV.1710.05312. URL: https://arxiv.org/abs/1710.05312 (cit. on p. 25).

[39] Z. Yi and K Tan. "Convergence Analysis of Recurrent Neural Networks". In: 13 (2004-01). DOI: 10.1007/978-1-4757-3819-3_1 (cit. on p. 25).

[40] L. Medsker, D. L. Jain, and B. R. L. N. Y. Washington. *RECURRENT NEURAL NETWORKS E dited by Design and Applications*. 2001 (cit. on p. 25).

[41] R. F. L. Neves et al. *An Overview of Deep Learning Strategies for Time Series Prediction Electrical and Computer Engineering Examination Committee*. 2018 (cit. on pp. 26–28).

[42] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016-11. URL: https://books.google.pt/books?id=omivDQAAQBAJ&lpg=PR5&ots=MNT2iopFQQ&dq=Deep\%20Learning\%20(Adaptive\%20Computation\%20and\%20Machine\%20Learning\%20series)&lr&hl=pt-PT&pg=PA191#v=onepage&q=Deep\%20Learning\%20(Adaptive\%20Computation\%20and\%20Machine\%20Learning\%20series)&f=false (visited on 2023-01-27) (cit. on p. 26).

[43] L. Medsker, D. L. Jain, and B. R. L. N. Y. Washington. *RECURRENT NEURAL NETWORKS E dited by Design and Applications*. 2001 (cit. on p. 26).

[44] K. Cho et al. "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation". In: (2014-06). URL: http://arxiv.org/abs/1406.1078 (cit. on pp. 28, 29).

[45] A. Creswell et al. "Generative Adversarial Networks: An Overview". In: *IEEE Signal Processing Magazine* 35.1 (2018), pp. 53–65. DOI: 10.1109/MSP.2017.2765202 (cit. on p. 29).

[46] *Overview of GAN Structure | Generative Adversarial Networks*. Google Developers, 2019. URL: https://developers.google.com/machine-learning/gan/gan_structure (cit. on pp. 29, 30).

[47] D. Bank, N. Koenigstein, and R. Giryes. "Autoencoders". In: (2020-03). URL: http://arxiv.org/abs/2003.05991 (cit. on p. 30).

[48] S. J. Ryu et al. "Feature-Based Hand Gesture Recognition Using an FMCW Radar and its Temporal Feature Analysis". In: *IEEE Sensors Journal* 18 (18 2018-09), pp. 7593–7602. ISSN: 1530437X. DOI: 10.1109/JSEN.2018.2859815 (cit. on p. 31).

[49] Y. Sun et al. "Gesture Classification with Handcrafted Micro-Doppler Features using a FMCW Radar". In: Institute of Electrical and Electronics Engineers Inc., 2018-08. ISBN: 9781538617250. DOI: 10.1109/ICMIM.2018.8443507 (cit. on p. 31).

[50] S. A. Shah and F. Fioranelli. "Human Activity Recognition: Preliminary Results for Dataset Portability using FMCW Radar". In: Institute of Electrical and Electronics Engineers Inc., 2019-09. ISBN: 9781728126609. DOI: 10.1109/RADAR41533.2019.171307 (cit. on p. 32).

[51] K. Han and S. Hong. "Phase-extraction method with multiple frequencies of FMCW radar for human body motion tracking". In: *IEEE Microwave and Wireless Components Letters* 30 (9 2020-09), pp. 927–930. ISSN: 15581764. DOI: 10.1109/LMWC.2020.3010262 (cit. on p. 33).

[52] R. Cruz, A. Furtado, and R. Oliveira. "Assessment of Feature Selection for Context Awareness RF Sensing Systems". In: vol. 2022-June. Institute of Electrical and Electronics Engineers Inc., 2022. ISBN: 9781665482431. DOI: 10.1109/VTC2022-Spring54318.2022.9860673 (cit. on p. 33).